CS 335 - Fall 2026: Assignment 1

Instructor: Leili Rafiee Sevyeri Email: Leili.rafiee.sevyeri@uwaterloo.ca

Due: Monday, Oct 5, 2026, 5:00pm

Submit all components of your solutions (written/analytical work, code/scripts/notebooks, figures, plots, etc.) to Crowdmark in PDF form in the section for each question.
You must also separately submit a single zip file containing any and all code/notebooks/scripts you write to the A1 DropBox on LEARN, in runnable format (that is, .ipynb).
For full marks, you must show your work and adequately explain your logic!

1. (6 marks) No-arbitrage option pricing.

   Suppose the current price of a stock is \$18. Consider a two state tree model defined such that after 3 months, the stock price may increase to \$19 with a 40% probability or decrease to \$16 with a 60% probability. Consider a European call option, based on this stock, with strike price K=\$17 and expiry time $T = 3$ months. Assume the risk-free interest rate is 9%.

   (a) (3 marks) Construct a risk-free portfolio and use it to determine the present no-arbitrage value (fair value) of the option (i.e., at time $t = 0$).

   Solution:

   The payoff in the "up" case is $\max(S_{up} - K, 0) = \max(19 - 17, 0) = 2$ and in the "down" case is $\max(S_{dn} - K, 0) = \max(16 - 17, 0) = 0$. Let's determine the number of shares to form a risk-free portfolio. $\Pi_{up} = \delta S - V = \delta(19) - 2$ and $\Pi_{dn} = \delta S - V = \delta(16) - 0$. Equating these we find $19\delta - 2 = 16\delta$ or $\delta = 2/3$. We can then determine the portfolio value to be $\Pi_{T=3} = (2/3)(16) = 10.667$. Discounting this back to time $t = 0$ based on the interest rate of 9% we get the initial value $\Pi_{t=0} = e^{-(0.09)(3/12)}10.667 \approx 10.429$. From this we can recover the initial ($t = 0$) option price using the definition of the portfolio again: $\Pi = \delta S - V$ so $10.429 = (2/3)(18) - V$ implying that $V \approx 1.571$. This is the no arbitrage price for the option at $t = 0$.

   (b) (3 marks) Suppose such an option is priced on the market at $V^* = 2.0$ (i.e., *higher* than the no-arbitrage price). Describe the details for a strategy that will result in a guaranteed profit (in present value, i.e., at $t = 0$) and determine the value of that profit (per option), somewhat similar to what we did in class. (As usual in this course, we will assume short-selling is allowed and the no-arbitrage principle holds.)

   Solution:

   We are given that $V^* = 2.0$ (i.e., > 1.571). (One could also do it for the general case of generic V^*, but the question considers the specific case.) In essence, we will do the reverse of the example that was considered in the lecture. (Equivalent solutions are acceptable.)

   We will hedge as follows: Buy $\delta = 2/3$ shares of the stock (by borrowing money from the bank), and short sell one call option at the stated price of $V^* = 2.0$. Thus we effectively have cash in the bank $= -(2/3) \times 18 + 2.0 = -10$ (i.e., we have borrowed 10 from the bank.)

   3 months later, we owe the bank the original amount we borrowed plus interest, so effectively we have cash of $-10e^{(0.09\times(3/12))}(\approx -10.228)$. Now, either the stock went up to 19 (in which case the option has a value/payoff of 2) or down to 16 (and the option is worth 0). Let's work out the value at the expiry.

In the first case, the call option costs us 2 to pay out, so cash $= -10\exp(0.09 \times (3/12)) - 2$. We also hold stock whose value is now $2/3 \times 19 \approx 12.667$. Our net value is:

$$\begin{aligned} & -10\exp(0.09 \times (3/12)) - 2 + 12.667 && (1)\\ & = -10\exp(0.0225) + 10.667 && (2)\\ & = \exp(0.0225)(-10 + 10.667\exp(-0.0225) && (3)\\ & = \exp(0.0225)(-10 + 10.429) && (4)\\ & = \exp(0.0225)(0.429) && (5)\\ & = \exp(0.0225)(V^* - V) && (6) \end{aligned}$$

This is the value at the expiry, so the present value (i.e., before interest) is 0.429, the difference of V and V^*.

In the other case, the call option costs us 0 to pay out, so we still have cash $= -10\exp(0.0225)$. The stock price is 16 at this point, so our stock holding is worth $2/3 \times 16 = 10.667$. Net holdings are:

$$-10\exp(0.0225) + 10.667 = \exp(0.0225)(0.429) = \exp(0.0225)(V^* - V) \qquad (7)$$

which is exactly as in the preceding calculation. Thus, in either case, we turn a profit, which is the *present value* of the difference $V^* - V = 2.0 - 1.571 \approx 0.429$.

2. (10 marks) Coding: Convergence of a walk on a discrete lattice.

Consider a random walk on an N step lattice on $t \in [0, T]$ with the parameters

$$\begin{aligned} X_{n+1} &\rightarrow X_n + \sigma\sqrt{\Delta t}\ ; \quad \text{with probability} \quad p \\ X_{n+1} &\rightarrow X_n - \sigma\sqrt{\Delta t}\ ; \quad \text{with probability} \quad 1-p \\ p &= \frac{1}{2}\left[1 + \frac{\alpha}{\sigma}\sqrt{\Delta t}\right] \\ \Delta t &= \frac{T}{N}\ , \end{aligned}$$

which converges to the solution of the SDE

$$dX = \alpha\ dt + \sigma\ dZ\ ,$$

as $\Delta t \to 0$. The exact solution to this SDE is such that the exact density (probability distribution) of $X(T)$ is a normal density with mean $X_{init} + \alpha T$ and standard deviation $\sigma\sqrt{T}$, assuming the initial value of X is X_{init}.

Verify this experimentally via numerical simulation of many lattice walks, by preparing a Jupyter/Python notebook to carry out the following. Perform a lattice walk simulation, using the parameters given in Table 1. Starting at $X_0 = X_{init}$, the value of X is then moved up with probability p or down with probability $(1-p)$. A *uniformly* distributed random number on $[0, 1]$ should be generated and used to determine the actual move taken. At the next lattice node (i.e., time step), this process is repeated, until we obtain the final position of X after N steps, X_N, at $t = T$. This is a single simulation outcome; we perform many such simulations to produce many (possibly) different outcomes. Because each step moves up or down a fixed amount, the set of all possible outcomes for this discrete lattice walk is

$$X_N^j = X_{init} + \left\{(2j - N)\sigma\sqrt{\Delta t}\right\} \qquad j = 0, \ldots, N.$$

As indicated in Table 1, you should perform the above process first for 1000 simulations, then separately for 10,000 simulations, and finally for 100,000 simulations. In each case, print out the number of simulations and the corresponding mean and standard deviation of the computed values of X_N from the simulations in that set. Also, print out the anticipated exact mean and standard deviation for comparison.

Next, for each set of simulations, we would like to generate a histogram of the observed *probability density* versus $X(T)$ as described below.

Care must be taken in scaling this histogram to get the correct probability density. Let $p(x)$ be the probability density, and let $\text{bin}_{[a,b]}$ be the bin on an interval $[a, b]$ of the x axis. Then

$$\begin{aligned} \int_a^b p(x)\, dx &\simeq p((a+b)/2)(b-a) \\ &\simeq \frac{\text{Number of occurrences in bin}_{[a,b]}}{\text{Total number of occurrences}} \end{aligned}$$

so that

$$p((a+b)/2) \simeq \frac{1}{(b-a)} \frac{\text{Number of occurrences in bin}_{[a,b]}}{\text{Total number of occurrences}}$$

Use $N+1$ bins, with the bin boundaries falling half-way between the lattice nodes at timestep N. Use matplotlib's *bar* function to display your probability density plot constructed as described above. (Alternatively, if you prefer, Matplotlib's *hist* or Numpy's *histogram* functionality can automate some of this process, if you give it appropriately chosen values for the range, bin width, and density parameters.) Be sure to include appropriate labeling on your plots (axis labels, title, legend if necessary).

Then, for comparison, over top of each of your histograms, generate a standard (line) plot of the true normal distribution with mean $X_{init} + \alpha T$ and standard deviation $\sigma\sqrt{T}$, over the same range (in a different color). The Scipy function *stats.norm.pdf* can be used to evaluate a true normal distribution function. Thus each of your three plots will look something roughly along the lines of Figure 1 (albeit generated with different data).

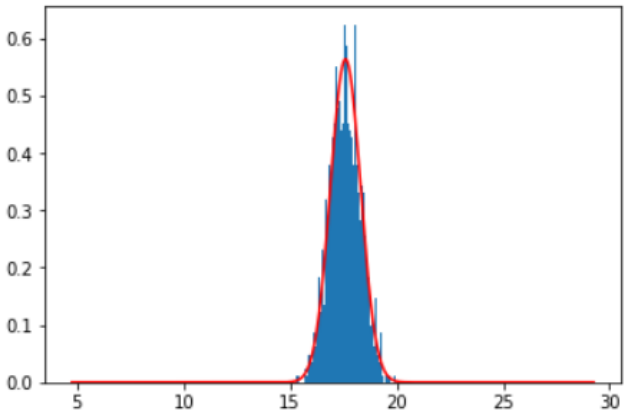


Figure 1: A histogram of an approximate probability density function, with a normal distribution plotted on top.

Your code must be *vectorized* for efficiency, i.e., prefer vector operations instead of scalar operations. As a very simple example of vectorization, if I have a Numpy vector V and I want to

T	1.75
σ	0.3
α	0.4
X_{init}	17
Number of simulations	1000, 10000, 100000
Number of timesteps (N)	300

Table 1: *Data to be used in your simulations of discrete walks on a lattice.*

add 1 to every entry of the vector, I can simply write the vector operation $V = V + 1$, instead of using a slower for loop over index i and separately computing $V[i] = V[i] + 1$ for each entry. Similarly, many Numpy functions work on vectors rather than single numbers.

Specifically, your code for the lattice walk should only explicitly loop over the timesteps, not over the individual simulations - for each time step, the updates for all the simulations (within a set) should be computed together using vector operations. Also, you should not store the entire lattice at once, but rather only one or two columns of it, to save on memory. The example lattice walk code in the course notes is likely to be *very* helpful, since it illustrates both vectorization and lattice walk implementation! (Course notes, page 17).

In addition to the NumPy functions mentioned previously above, you may also find the *mean*, *std*, and *random.uniform* functions useful.

In a markdown cell in your Jupyter notebook, describe what you observe about your results (plots and statistics) as the number of simulations increases from 1000 to 100,000.

3. (6 marks) Coding: Brownian motion and Ito calculus.

   Let

$$\begin{aligned} \Delta Z_i &= \phi(t_i)\sqrt{\Delta t} \\ \phi(t_i) &\sim N(0,1) \\ Z(t_0 = 0) &= 0 \\ Z(t_i) &= \sum_{j=0}^{i-1} \Delta Z_j \\ t_i &= i\ \Delta t \end{aligned}$$

   and

$$\begin{aligned} I(T, \Delta t) &= \sum_{i=0}^{N-1} (\Delta Z_i)^2 \\ \Delta t &= \frac{T}{N}. \end{aligned}$$

   Write a Jupyter/Python notebook to evaluate $I(T, \Delta t)$. For $T = 2$, for fixed Δt, compute the value of $I(T, \Delta t)$. Use 70000 paths, and compute the mean and variance of the result, for $N =$ 100, 200, 400, 800, 1600, 3200, 6400 timesteps. Vectorize your code so that there is only one explicit for loop over the timesteps, for a given value of N.

Write code to generate a simple text-based table of mean and variance of $I(T, \Delta t)$ for the different values of Δt. (No fancy formatting is required! You can just use Python's `print` function to generate each row, with appropriate arguments.)

Plot a graph of $var(I(T, \Delta t))$ versus Δt, with appropriate labels. In a markdown cell of your Jupyter notebook, explain mathematically what you observe.

4. (5 marks) Ito calculus.

   (a) (3 marks) Let $X(t)$ and $Y(t)$ be two stochastic processes, such that

$$\begin{aligned} dX &= \mu_X(X(t), t)dt + \sigma_X(X(t), t)dZ_X \\ dY &= \mu_Y(Y(t), t)dt + \sigma_Y(Y(t), t)dZ_Y \end{aligned}$$

   with $dZ_X(t)$, $dZ_Y(t)$ being the increments for two distinct Wiener processes, $Z_X(t)$ and $Z_Y(t)$. Let $X(t_i) = X_i$ and $Y(t_i) = Y_i$. Show that

$$(X_{i+1} - X_i)(Y_{i+1} - Y_i) = X_{i+1}Y_{i+1} - X_iY_i - X_i(Y_{i+1} - Y_i) - Y_i(X_{i+1} - X_i) \ .$$

   Then, using the definition of the Ito integral which is the limit of a discrete sum, show that

$$\int_0^t X(s)dY(s) \ = \ [XY]_0^t - \int_0^t Y(s)dX(s) - \int_0^t dX(s)dY(s).$$

   Solution:
   First, consider the right-hand side:

$$X_{i+1}Y_{i+1} - X_iY_i - X_i(Y_{i+1} - Y_i) - Y_i(X_{i+1} - X_i).$$

   Expanding gives

$$X_{i+1}Y_{i+1} - X_iY_i - X_iY_{i+1} + X_iY_i - X_{i+1}Y_i + X_iY_i,$$

   and after cancellation,

$$X_{i+1}Y_{i+1} - X_iY_{i+1} - X_{i+1}Y_i + X_iY_i.$$

   Now, the left-hand side is

$$(X_{i+1} - X_i)(Y_{i+1} - Y_i),$$

   which expands to

$$X_{i+1}Y_{i+1} - X_iY_{i+1} - X_{i+1}Y_i + X_iY_i.$$

   Thus, the two sides are equal.
   It is useful to rearrange this identity as

$$X_{i+1}Y_{i+1} - X_iY_i = X_i(Y_{i+1} - Y_i) + Y_i(X_{i+1} - X_i) + (X_{i+1} - X_i)(Y_{i+1} - Y_i).$$

   Now sum both sides from $i = 0$ to $N - 1$:

$$\sum_{i=0}^{N-1} (X_{i+1}Y_{i+1} - X_iY_i) = \sum_{i=0}^{N-1} X_i(Y_{i+1} - Y_i) + \sum_{i=0}^{N-1} Y_i(X_{i+1} - X_i) + \sum_{i=0}^{N-1} (X_{i+1} - X_i)(Y_{i+1} - Y_i).$$

The sum on the left is a telescoping sum:

$$(X_1Y_1 - X_0Y_0) + (X_2Y_2 - X_1Y_1) + \cdots + (X_NY_N - X_{N-1}Y_{N-1}),$$

so all the intermediate terms cancel, leaving

$$X_NY_N - X_0Y_0.$$

Therefore,

$$\sum_{i=0}^{N-1} X_i(Y_{i+1} - Y_i) = X_NY_N - X_0Y_0 - \sum_{i=0}^{N-1} Y_i(X_{i+1} - X_i) - \sum_{i=0}^{N-1} (X_{i+1} - X_i)(Y_{i+1} - Y_i).$$

Now take the limit as $\Delta t \to 0$. By the definition of the Ito integral,

$$\sum_{i=0}^{N-1} X_i(Y_{i+1} - Y_i) \longrightarrow \int_0^t X(s)\, dY(s),$$

and similarly,

$$\sum_{i=0}^{N-1} Y_i(X_{i+1} - X_i) \longrightarrow \int_0^t Y(s)\, dX(s).$$

Also,

$$X_NY_N - X_0Y_0 \longrightarrow X(t)Y(t) - X(0)Y(0).$$

We write this more compactly as

$$[XY]_0^t = X(t)Y(t) - X(0)Y(0),$$

which is the usual notation for evaluating the quantity XY at the upper and lower limits. Hence,

$$\int_0^t X(s)\, dY(s) = [XY]_0^t - \int_0^t Y(s)\, dX(s) - \int_0^t dX(s)\, dY(s)$$

This is the Ito integration rule.

(b) (2 marks) Let $Z(t)$ be a stochastic process satisfying $dZ = \phi\sqrt{dt}$, and assume $Z(0) = 0$. Using the result in part (a), show that (assuming Ito calculus)

$$\int_0^t Z(s)\ dZ(s) \quad = \quad \frac{1}{2}Z(t)^2 - \frac{t}{2}\ .$$

Solution: We use the IBP rule where X=Y, yielding:

$$2\int_0^t ZdZ = [Z^2]_0^t - \int_0^t dZ^2 \tag{8}$$

$$\int_0^t ZdZ = \frac{1}{2}([Z^2]_0^t - \int_0^t dZ^2) \tag{9}$$

$$\int_0^t ZdZ = \frac{1}{2}([Z^2]_0^t - \int_0^t dt) \tag{10}$$

$$\int_0^t ZdZ = \frac{1}{2}(Z(t)^2 - Z(0)^2 - \int_0^t dt) \tag{11}$$

In the case where $Z(0) = 0$, we get the requested result.

5. (10 marks) Ito calculus and Ito's Lemma.

Suppose dZ is the increment of a Wiener process and

$$dX = \mu(X(t), t)dt + \sigma(X(t), t)dZ.$$

(a) (6 marks) If

$$F(X, t) = X^4 e^{-t} + 3X \cos t,$$

what is $dF(X, t)$ (interpreted in the sense of Ito calculus)?

Solution:

$$\frac{\partial F}{\partial X} = 4X^3 e^{-t} + 3\cos t$$
$$\frac{\partial^2 F}{\partial X^2} = 12X^2 e^{-t}$$
$$\frac{\partial F}{\partial t} = -X^4 e^{-t} - 3X \sin t$$

Then

$$dF = (\mu F_X + F_{XX}\frac{\sigma^2}{2} + F_t)dt + F_X \sigma dZ \tag{12}$$
$$= \Big[\mu(4X^3 e^{-t} + 3\cos t) + 6X^2 e^{-t}\sigma^2 - X^4 e^{-t} - 3X \sin t\Big]dt + (4X^3 e^{-t} + 3\cos t)\sigma\, dZ. \tag{13}$$

(b) (4 marks) Using dX as defined above, and expanding, simplify

$$\int_0^T (dX)^2$$

as much as possible. The integral is to be interpreted in the Ito calculus sense. (Note that $\sigma(X(t), t)$ is a function of t, not a constant, so you will not be able to fully solve the integral.)

Solution:

$$\int_0^T (dX)^2 = \int_0^T (\mu dt + \sigma dZ)^2 \tag{14}$$
$$= \int_0^T \mu^2 dt^2 + 2\mu\sigma dt dZ + \sigma^2 dZ^2 \tag{15}$$
$$= \int_0^T \mu^2 dt^2 + 2\mu\sigma\phi dt^{3/2} + \sigma^2 dt \tag{16}$$

where above we have used the fact that $dZ^2 \to dt$ as $dt \to 0$ and $dZ = \phi\sqrt{dt}$. Using Big O notation, as in the Ito's lemma derivation, we group the higher order terms of dt. In the limit, only the first order dt term remains.

$$\int_0^T (dX)^2 = \int_0^T [\sigma(X(t), t)]^2 dt + O(dt^{1.5}) \tag{17}$$
$$= \int_0^T [\sigma(X(t), t)]^2 dt \tag{18}$$

That's it.